

A Practical Guide to Flow-based generative model

TOPIC -- FLOW-BASED GENERATIVE MODEL

-- 01 --

$$p_1(z_1) = p_0(z_0) \left| \det \left(\frac{df_1(z_0)}{dz_0} \right) \right|^{-1} \quad \{\displaystyle p_{\{1\}}(z_{\{1\}})=p_{\{0\}}(z_{\{0\}})\left|\det \left(\frac{df_{\{1\}}(z_{\{0\}})}{dz_{\{0\}}}\right)^{-1}\right|\}$$

-- 02 --

Simplex calibration transform A calibration transform, $f_{\text{cal}} : \Delta^{n-1} \rightarrow \Delta^{n-1}$, which is sometimes used in machine learning for post-processing of the (class posterior) outputs of a probabilistic n -class classifier, uses the softmax function to renormalize categorical distributions after scaling and translation of the input distributions in log-probability space. For $\mathbf{p}, \mathbf{q} \in \Delta^{n-1}$ and with parameters, $\mathbf{a} \neq \mathbf{0}$ and $\mathbf{c} \in \mathbb{R}^n$ the transform can be specified as:

-- 03 --

$$\arg \max_{\theta} \sum_{i=0}^N \log \pi_{\theta}(x_i) \quad \{\displaystyle \{\underset{\{\theta\}}{\operatorname{arg\,max}}\} \sum_{i=0}^N \log p_{\{\theta\}}(x_{\{i\}})\}$$

-- 04 --

$$\text{volume}(\mathbf{U}) = |\det(\mathbf{D}\mathbf{E}'\mathbf{E}\mathbf{D})| = |\det(\mathbf{E}'\mathbf{E})| |\det \mathbf{D}| = n \prod_{i=1}^{n-1} |\det \mathbf{D}_i| \quad \{\displaystyle \operatorname{volume}(\mathbf{U}) = \sqrt{|\det(\mathbf{DE}'\mathbf{ED})|} = \sqrt{|\det(\mathbf{E}'\mathbf{E})| |\det \mathbf{D}|} = \sqrt{|\det \mathbf{E}| |\det \mathbf{D}|} = \sqrt{n} \prod_{i=1}^{n-1} |\det \mathbf{D}_i|}\}$$

-- 05 --

Real Non-Volume Preserving (Real NVP) The Real Non-Volume Preserving model generalizes NICE model by: $\mathbf{x} = [\mathbf{x}_1 \times \mathbf{x}_2] = \mathbf{f}_{\theta}(\mathbf{z}) = [\mathbf{z}_1 \exp(\mathbf{z}_1) \odot \mathbf{z}_2] + [\mathbf{0} \mid \mathbf{m}_{\theta}(\mathbf{z}_1)] \quad \{\displaystyle \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \mathbf{f}_{\theta}(\mathbf{z}) = \begin{bmatrix} z_1 \exp(z_1) \\ z_2 \end{bmatrix} \odot \begin{bmatrix} z_1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ m_{\theta}(z_1) \end{bmatrix}}\}$

-- 06 --

$$R_{\text{aff}}(\mathbf{x}; \mathbf{M}, \mathbf{c}) = R_{\text{lin}}(\mathbf{x}; \mathbf{M} + \mathbf{c} \mathbf{x}') = \frac{\det \mathbf{M}}{(1 + \mathbf{x}' \mathbf{M}^{-1} \mathbf{c})} \mathbf{M} \mathbf{x} + \mathbf{c}$$

-- 07 --

To understand the geometric interpretation of the factor $\sqrt{n}$, see the example for the 1-simplex in the diagram at right. The differential volume element at the transformation output ($\mathbf{q} \in \Delta^{n-1}$), is the parallelotope, $V = f(\mathbf{U}) = \mathbf{F} \mathbf{p} \in D /$
 $V = f(\mathbf{U}) = \mathbf{F}_{\mathbf{p}} \mathbf{E} \mathbf{D}$, where $\mathbf{F}_{\mathbf{p}}$ is the n -by- n Jacobian of f at $\mathbf{p} = \mathbf{e}(\mathbf{p})$. Its volume is:

-- 08 --

The trace can be estimated by "Hutchinson's trick": Given any matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$, and any random $\mathbf{u} \in \mathbb{R}^n$ with $\mathbb{E}[\mathbf{u} \mathbf{u}^T] = \mathbf{I}$, we have $\mathbb{E}[\mathbf{u}^T \mathbf{W} \mathbf{u}] = \text{tr}(\mathbf{W})$. (Proof: expand the expectation directly.) Usually, the random vector is sampled from $N(0, \mathbf{I})$ (normal distribution) or $\{\pm 1/\sqrt{2}\}^n$ (Rademacher distribution). When f is implemented as a neural network, neural ODE methods would be needed. Indeed, CNF was first proposed in the same paper that proposed neural ODE. There are two main deficiencies of CNF, one is that a continuous flow must be a homeomorphism, thus preserve orientation and ambient isotopy (for example, it's impossible to flip a left-hand to a right-hand by continuous deforming of space, and it's impossible to turn a sphere inside out, or undo a knot), and the other is that the learned flow f might be ill-behaved, due to degeneracy (that is, there are an infinite number of possible f that all solve the same problem). By adding extra dimensions, the CNF gains enough freedom to reverse orientation and go beyond ambient isotopy (just like how one can pick up a polygon from a desk and flip it around in 3-space, or unknot a knot in 4-space), yielding the "augmented neural ODE". Any homeomorphism of $\mathbb{R}^n$ can be approximated by a neural ODE operating on $\mathbb{R}^{2n+1}$, proved by combining Whitney embedding theorem for manifolds and the universal approximation theorem for neural networks. To regularize the flow f , one can impose

regularization losses. The paper proposed the following regularization loss based on optimal transport theory: $\lambda_K \int_0^T \int \|\mathbf{f}(\mathbf{z}_t, t) - \mathbf{f}(\mathbf{z}_{t'}, t')\|^2 dt + \lambda_J \int_0^T \int \|\nabla_{\mathbf{z}} \mathbf{f}(\mathbf{z}_t, t) - \nabla_{\mathbf{z}} \mathbf{f}(\mathbf{z}_{t'}, t')\|^2 dt$ where $\lambda_K, \lambda_J > 0$ are hyperparameters. The first term punishes the model for oscillating the flow field over time, and the second term punishes it for oscillating the flow field over space. Both terms together guide the model into a flow that is smooth (not "bumpy") over space and time.

-- 09 --

INPUT. dataset $x_{1:n}$, normalizing flow model $f_{\theta}(\cdot)$, p_0 . SOLVE. $\max_{\theta} \sum_j \log p_{\theta}(x_j)$ by gradient descent
RETURN. $\hat{\theta}$

-- 10 --

If f_{gcal} is to be used as a calibration transform, further constraint could be imposed, for example that A be positive definite, so that $(A\mathbf{x})' \mathbf{x} > 0$, which avoids direction reversals. (This is one possible generalization of $a > 0$ in the f_{cal} parameter.) For $n = 2$, $a > 0$ and A positive definite, then f_{cal} and f_{gcal} are equivalent in the sense that in both cases, $\log \frac{p_1}{p_2} \mapsto \log \frac{q_1}{q_2}$ is a straight line, the (positive) slope and offset of which are functions of the transform parameters. For $n > 2$, f_{gcal} does generalize f_{cal} . It must however be noted that chaining multiple f_{gcal} flow transformations does not give a further generalization, because: